

2.3.2 Number of Conditions

IF پروسیسر سے True/False پوچھتی ہے اس لیے زیادہ شاخیں کارکردگی گھٹاتی ہیں۔

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 07 of 16

2.3.2 Number of Conditions

LEARN IT

Conditions (IF statements) are decision points. Every condition forces the CPU to evaluate True or False, so excessive branching reduces performance.

- **Computational cost.** Each IF is extra work, even if the rest of the algorithm is simple.
- **Nested conditions.** IF inside IF often means a brute-force approach. It hurts both readability and speed.
- **Short-circuiting** (the optimisation goal). Stop evaluating as soon as the result is known. In a logical AND, if the first part is False the second part is skipped. In a logical OR, if the first part is True the second part is skipped.

WORKED EXAMPLE

Worked example: cheap test first. You only divide if the denominator is not zero. Place the cheap, safe test on the left so Python never evaluates the dangerous part.

```
x = 0
y = 10
if x != 0 and (y / x) > 2:
    print("ratio is large")
else:
    print("skipped safely")
```

OUTPUT

```
skipped safely
```

$x \neq 0$ is False, so AND short-circuits and y / x is never attempted. If you wrote the division first, the program would crash. Same idea in the soldier algorithm: a False height test skips Step 6.

REMEMBER IT

Put the **cheapest / most likely False** test first in an AND, and the **most likely True** test first in an OR. That is short-circuiting used on purpose.

COMMON MISTAKE

Mistake: writing a chain of nested IFs when one compound condition would do (as in the velocity example in 2.2.2).

Fix: flatten with AND/OR, and rely on short-circuiting instead of extra layers.

اردو میں سمجھیں

IF پروسیسر سے True/False پوچھتی ہے اس لیے زیادہ شاخیں کارکردگی گھٹاتی ہیں۔ اندرونی IF اکثر brute-force ہوتی ہے۔ شارٹ سرکٹ (Short-circuit) کا مطلب ہے جیسے ہی جواب معلوم ہو جانچ روک دی جائے، مثلاً AND میں پہلی شرط False ہو تو دوسری نہ چلے۔

CHECK YOURSELF

1. Why do extra IF statements reduce performance?
2. What does nested IF often indicate?
3. In $x \neq 0$ and $(y / x) > 2$ with $x = 0$, why is there no division-by-zero?

Answers: 1. Each condition must be evaluated True/False by the CPU. 2. A brute-force approach (and it reduces readability and speed). 3. AND short-circuits: the first part is False, so the division is never evaluated.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Number of conditions affects:

- a. How many decisions (and often branches) you pay for
- b. Only the font
- c. Only RAM manufacturing
- d. Only UX colour

MCQ 2. A nested if inside a loop can make work:

- a. Depend on data (sometimes skip inner work)
- b. Always n^2
- c. Always $O(1)$
- d. Always crash

MCQ 3. Short-circuit and/or can:

- a. Skip later tests
- b. Sort the array
- c. Open a file
- d. Draw a pie

MCQ 4. Too many rare special cases:

- a. Hurt clarity and can hurt time if mis-ordered
- b. Always reduce Big O class
- c. Always increase RAM
- d. Always fix UX

MCQ 5. Binary search's conditions:

- a. Few, but they cut n in half — very efficient
- b. n^2
- c. Are UX heuristics
- d. Are SQL joins

MCQ 6. Counting conditions is part of the:

- a. Efficiency evaluation framework
- b. HCI Nielsen list only
- c. ER cardinality only
- d. MVP canvas only

Extra short questions

1. What is a "condition" in this chapter's sense?
2. How can conditions save time?
3. How can they waste time?
4. Link to readability.

Extra long question

1. Two versions of search: (A) no break (B) break when found. Compare conditions, time, traces on a hit at position 2 of 10.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. A true/false test the algorithm performs (if, loop test, while).
2. Early exit: if found, break; if $x < 0$, skip expensive work.
3. Re-testing the same expensive predicate; poorly ordered or/and.
4. A tower of nested ifs is slow to understand even if $O(n)$.

Long

1. A always 10 tests. B 2 tests. Same Big O class worst-case still $O(n)$, but average better. Show trace tables.

Next lecture: 2.3.3 Number of Repetitions